

TESTE PRÁTICO — DESENVOLVEDOR .NET SÊNIOR

Order Management

Perguntas & respostas

Roteiro completo para apresentação e entrevista técnica, com perguntas, respostas revisadas e evidências da implementação.

Escopo

Clean Architecture, CQRS, domínio, persistência, API, testes, observabilidade e entrega.

Revisão

Build, testes, cobertura, migrations, frontend, Docker e fluxo HTTP validados em 30/08/2026.

REVISÃO TÉCNICA INDEPENDENTE

Solução alinhada ao enunciado

A implementação atende aos requisitos obrigatórios e desejáveis verificados. Não foram encontradas divergências críticas entre o enunciado, o guia de respostas e o código executável.

0 erros e warnings no build Release	49/49 testes .NET aprovados	94,8% linhas em Application + Domain (unitários)	100% branches em Application + Domain (unitários)
---	---	--	---

Checklist de aderência

Área	Evidência observada	Status
Arquitetura	Domain, Application, Infrastructure e API com direção de dependências correta	CONFORME
Domínio	Invariantes, transições e TotalAmount concentrados no agregado	CONFORME
CQRS + MediatR	Commands e queries separados; validação e logging no pipeline	CONFORME
Persistência	EF Core + SQLite; migration inicial aplicada no startup e sem alterações pendentes	CONFORME
Segurança	Login fixo solicitado, JWT validado e rotas de pedidos protegidas	CONFORME
Testes	41 unitários e 8 de integração aprovados; cobertura unitária confirmada	CONFORME
Entrega	Dockerfiles, Compose, README, Swagger, Serilog, OpenTelemetry e SonarQube	CONFORME
Frontend adicional	Build Vite, 4 testes do worker e integração real com a API	CONFORME

Validações executadas

Build e formatação .NET; suíte unitária e de integração com cobertura; verificação de migrations; testes e build do frontend; validação do Compose; build das imagens de API e frontend; e smoke test em contêiner para health, login, criação, listagem, consulta, cancelamento e conflito no segundo cancelamento.

Pontos de atenção já assumidos no guia

SQLite, credenciais fixas, chave local, migrations no startup, ausência de concorrência otimista, idempotência e moeda são limitações proporcionais ao teste. O próprio guia as apresenta corretamente como decisões de escopo e itens a evoluir em produção.

Base da revisão: enunciado “Teste Prático — Desenvolvedor .NET Senior”, [Instructions.md](#), [answers.md](#), código-fonte e artefatos executáveis do workspace em 30 de agosto de 2026.

MAPA DE CONTEÚDO

Guia completo para a apresentação

78 respostas organizadas por assunto, do resumo inicial às extensões prováveis ao vivo.

01 Resumo da solução em um minuto

03 Arquitetura e design

05 CQRS, MediatR e pipeline

07 API, autenticação e segurança

09 Logging, tracing e operação

11 Integração frontend

13 Perguntas difíceis e respostas honestas

15 Comandos úteis durante a entrevista

02 Perguntas indicadas no Instructions.md

04 Domínio

06 Persistência

08 Testes

10 Docker e entrega

12 Extensões prováveis ao vivo

14 Roteiro para demonstrar ao avaliador

Este arquivo reúne respostas baseadas no código entregue. A ideia não é decorar frases, mas usar os tópicos como roteiro e conseguir apontar a implementação correspondente durante a entrevista.

Resumo da solução em um minuto

Eu construí um monólito modular em .NET 10 com quatro projetos seguindo Clean Architecture. A API usa Minimal APIs e apenas traduz HTTP para commands e queries do MediatR. A Application coordena os casos de uso, valida entradas com FluentValidation e expõe abstrações específicas. O Domain contém o agregado `Order`, responsável pelas invariantes, transições de status e cálculo do total. A Infrastructure implementa SQLite/EF Core, JWT e o repositório. A API aplica migrations automaticamente, autentica com Bearer, padroniza erros com ProblemDetails e oferece Swagger, Serilog e OpenTelemetry. A solução possui 41 testes unitários e 8 de integração.

Perguntas indicadas no Instructions.md

PERGUNTA 01

Por que escolheu essa abordagem?

RESPOSTA

Escolhi um monólito modular porque existe apenas um contexto pequeno, gestão de pedidos, sem uma necessidade demonstrada de escala ou deploy independente. Microserviços trariam rede, consistência eventual, observabilidade distribuída e maior custo operacional prematuramente.

Usei Clean Architecture para fazer as dependências apontarem para as regras de negócio. Usei CQRS com MediatR porque foi exigido e porque os behaviors de validação e logging têm aplicação clara. Usei Minimal APIs porque há poucos endpoints finos; Controllers acrescentariam estrutura sem benefício proporcional neste escopo.

Minha preocupação foi aplicar cada padrão por um motivo concreto. Por exemplo, criei `IOrderRepository`, específico para o agregado, e não um `IRepository<T>` genérico.

PERGUNTA 02

Como organizou as camadas?

RESPOSTA

- `Domain`: `Order`, `OrderItem`, `OrderStatus`, value data e `DomainException`. Não depende de EF, MediatR, HTTP ou Infrastructure.
- `Application`: commands, queries, handlers, DTOs de saída, validators, behaviors e interfaces `IOrderRepository` e `IAuthenticationService`. Depende apenas de Domain.
- `Infrastructure`: `DbContext`, configurações do EF, migrations, repositório SQLite e emissão JWT. Implementa as interfaces da Application.
- `Api`: endpoints, contratos HTTP, autenticação Bearer, CORS, Swagger, ProblemDetails e composição das dependências.

O fluxo principal é `API -> Application -> Domain`. A Infrastructure também aponta para Application e Domain. O Domain nunca aponta para fora.

PERGUNTA 03

Onde posicionou as regras de negócio?

RESPOSTA

No agregado `Order` e na entidade `OrderItem`:

- `Order.Create` exige cliente e pelo menos um item;
- `OrderItem.Create` exige nome, quantidade e preço válidos;
- `Order.Cancel` aceita somente `Pending`;
- `Order.Confirm` aceita somente `Pending`;
- `TotalAmount` e o subtotal são calculados a partir dos itens.

FluentValidation também verifica entradas para produzir mensagens HTTP melhores, mas não substitui o domínio. Essa duplicação é intencional: validação de contrato melhora a experiência do cliente; invariantes no domínio garantem consistência qualquer que seja a origem da chamada.

PERGUNTA 04

Como garantiu testabilidade?

RESPOSTA

- Handlers dependem de interfaces pequenas, não de EF Core diretamente.
- `TimeProvider` é injetado, então data de criação e expiração podem ser determinísticas.
- Os handlers são pequenos e não conhecem HTTP.
- O domínio não depende de infraestrutura.
- Os unit tests usam `FakeOrderRepository` e stubs explícitos, sem banco e sem framework de mocks.
- Validators e os dois pipeline behaviors têm testes próprios.
- `WebApplicationFactory` testa o pipeline HTTP completo.
- Cada fixture de integração usa um arquivo SQLite exclusivo, preservando o comportamento real do provedor sem compartilhar o banco da aplicação.

Atualmente há 41 testes unitários, 8 de integração, aproximadamente 95% de cobertura de linhas em Application/Domain e 100% dos branches nessas duas camadas.

PERGUNTA 05

Quais trade-offs considerou?

RESPOSTA

Decisão	Benefício	Custo ou limite aceito
Monólito modular	Deploy e consistência simples	Não escala módulos separadamente
Minimal APIs	Pouca cerimônia	Muitos endpoints podem exigir organização adicional
MediatR	Pipeline uniforme e casos de uso isolados	Indireção e mais tipos
CQRS no mesmo banco	Separação de intenção sem complexidade distribuída	Não há modelos/bancos de leitura independentes
SQLite	Zero infraestrutura e boa avaliação local	Concorrência e operação inferiores a um banco servidor
Migration no startup	Ambiente inicia pronto	Réplicas podem disputar migrations em produção
JWT fixo	Atende ao teste de modo simples	Não há cadastro, refresh, revogação ou gestão de usuários
Total calculado	Uma única fonte de verdade	Listagens muito grandes recalculam a partir dos itens
Repositório específico	Intenção explícita	Uma interface a mais para manter
Erros via exceção	Endpoints e handlers claros	Exceções esperadas têm custo; <code>Result</code> seria alternativa

PERGUNTA 06

Como evoluiria a solução?

RESPOSTA

Eu evoluiria guiado por requisitos e métricas:

1. Adicionaria confirmação do pedido como command e endpoint, reutilizando `Order.Confirm`.
2. Introduziria filtros e, para grandes volumes, paginação por cursor.
3. Migraria para PostgreSQL quando concorrência, backup e operação exigissem.
4. Adicionaria concorrência otimista e idempotência.
5. Integraria um provedor de identidade e autorização por política.
6. Representaria `Money` /moeda e uma política de arredondamento no domínio.
7. Adicionaria eventos de domínio e outbox quando surgissem integrações externas.
8. Extrairia um serviço somente se escala, ownership ou ciclo de deploy justificassem.

PERGUNTA 07

O que faria diferente em produção?

RESPOSTA

- Secrets viriam de secret manager e teriam rotação.
- Autenticação usaria IdP/OIDC; não haveria senha fixa nem chave versionada.
- HTTPS seria obrigatório e o Swagger estaria desabilitado ou protegido.
- CORS permitiria somente domínios oficiais.
- PostgreSQL substituiria SQLite.
- Migrations rodariam uma vez em uma etapa controlada do deploy.
- Haveria rate limiting, auditoria, métricas, alertas e tracing via OTLP.
- Adicionaria health checks de dependências, readiness e liveness separados.
- Implementaria concorrência otimista, idempotency key e política de retry.
- Logs teriam retenção, correlação e redaction centralizada.
- Testes de contrato, carga, segurança e recuperação complementariam a suíte.

Arquitetura e design

PERGUNTA 08

Isso é realmente Clean Architecture?

RESPOSTA

Sim no aspecto essencial: a direção das dependências. O Domain é independente; a Application depende do Domain; a Infrastructure implementa portas da Application; a API é a composition root. Não afirmei que quantidade de projetos, sozinha, torna uma solução limpa.

PERGUNTA 09

Por que quatro projetos em vez de pastas em um único projeto?

RESPOSTA

Projetos criam fronteiras verificadas pelo compilador. Domain não consegue importar EF ou tipos da API por acidente. Para um código ainda menor, pastas poderiam bastar, mas o teste solicita camadas explícitas e o custo de quatro projetos permanece baixo.

PERGUNTA 10

Por que monólito modular e não microserviços?

RESPOSTA

Não há evidência de limites de negócio, equipes ou escala diferentes. Um serviço por operação seria um distributed monolith. Eu começaria com limites internos fortes e extrairia somente quando uma necessidade concreta pagasse o custo de rede, entrega, dados distribuídos e observabilidade.

PERGUNTA 11

Quando você extrairia um microserviço?

RESPOSTA

Quando um módulo exigisse escala muito diferente, tivesse equipe e deploy autônomos, precisasse de isolamento regulatório ou disponibilidade independente, e seu limite de dados estivesse claro. A extração viria acompanhada de contrato, ownership do banco, idempotência e observabilidade.

PERGUNTA 12

Por que Minimal APIs?

RESPOSTA

Os endpoints são poucos e finos. Eles fazem binding, criam command/query, chamam `ISender` e formatam status HTTP. Minimal APIs reduzem boilerplate sem comprometer separação. Eu migraria ou agruparia com Controllers se filtros, convenções, versionamento e políticas de MVC se tornassem mais vantajosos.

PERGUNTA 13

Os endpoints contêm lógica de negócio?

RESPOSTA

Não. A conversão de request HTTP para command e a escolha de `201`, `200` ou `204` são lógica de apresentação. Validação de contrato fica no pipeline e decisão de negócio fica no domínio.

PERGUNTA 14

Por que não usar um serviço `OrderService`?

RESPOSTA

Um serviço genérico tenderia a concentrar casos de uso diferentes. Commands/queries e handlers expressam cada intenção de forma pequena. Um domain service seria criado somente para uma regra que não pertencesse naturalmente a uma entidade ou value object.

PERGUNTA 15

Quais princípios SOLID aparecem aqui?

RESPOSTA

- SRP: endpoints, handlers, domínio, repositório e autenticação têm motivos distintos para mudar.
- DIP: Application depende de `IOrderRepository` e `IAuthenticationService`, não das implementações.
- OCP: é possível trocar SQLite ou JWT registrando outra implementação sem alterar handlers.
- ISP: interfaces possuem operações necessárias ao contexto, sem uma interface genérica extensa.

Domínio

PERGUNTA 16

Por que `Order` é a raiz do agregado?

RESPOSTA

Itens não têm ciclo independente neste contexto. Toda criação e alteração deve preservar a consistência do pedido como unidade. Por isso `Order` controla a coleção e `OrderItem.Create` é interno ao assembly do domínio.

PERGUNTA 17

Por que a coleção de itens é somente leitura?

RESPOSTA

Para impedir `order.Items.Add(...)` fora do agregado e evitar que alguém contorne invariantes. O EF Core materializa o campo privado `_items` por configuração de backing field.

PERGUNTA 18

Por que existem construtores privados?

RESPOSTA

O EF Core precisa materializar entidades, mas o código de negócio deve usar as fábricas controladas. Assim, uma criação normal não produz um pedido sem itens ou um item inválido.

PERGUNTA 19

Por que o total não é salvo no banco?

RESPOSTA

Ele é derivado de dados já persistidos. Persisti-lo criaria risco de divergência. Se medição real mostrasse custo de leitura, eu avaliaria uma projeção/materialized view ou coluna atualizada com uma estratégia consistente, sem abandonar a fonte de verdade do domínio.

PERGUNTA 20

Por que `decimal`?

RESPOSTA

`decimal` representa valores decimais de modo apropriado para dinheiro, ao contrário dos erros binários de `double`. O EF usa precisão `18,2`. Em produção eu também modelaria moeda e regra de arredondamento.

PERGUNTA 21

Por que status é salvo como texto?

RESPOSTA

O banco fica legível e uma reordenação dos membros do enum não muda o significado persistido. O custo é mais espaço do que um inteiro, irrelevante para este escopo.

PERGUNTA 22

O cancelamento é idempotente?

RESPOSTA

Não pela regra atual. O primeiro cancelamento de um `Pending` retorna `204`; o segundo encontra `Cancelled` e retorna `409`. Eu manteria isso porque o teste diz que apenas `Pending` pode ser cancelado. Se o contrato exigisse idempotência, faria `Cancelled -> Cancelled` retornar sucesso e documentaria essa semântica.

PERGUNTA 23

Existe problema de concorrência no cancelamento?

RESPOSTA

Em duas requisições realmente simultâneas, ambas podem ler `Pending` antes da gravação. SQLite serializa parte do acesso, mas não substitui controle de concorrência da entidade. Em produção eu adicionaria um concurrency token e trataria `DbUpdateConcurrencyException`, retornando `409` ou recarregando conforme o contrato.

PERGUNTA 24

Por que UTC e `TimeProvider`?

RESPOSTA

UTC evita que persistência e integrações dependam do fuso do servidor. `TimeProvider` oferece o relógio do sistema em produção e um relógio fixo nos testes, sem criar uma abstração própria.

CQRS, MediatR e pipeline

PERGUNTA 25

O que é CQRS nesta solução?

RESPOSTA

É a separação de modelos de intenção: commands alteram estado (`CreateOrder` , `CancelOrder`) e queries leem (`GetOrderById` , `ListOrders`). Não significa obrigatoriamente bancos separados nem consistência eventual. Aqui ambos usam o mesmo SQLite porque é a opção proporcional ao problema.

PERGUNTA 26

O que o MediatR resolve?

RESPOSTA

O endpoint conhece `ISender` , não o handler concreto. O MediatR localiza o handler e executa os behaviors registrados. Isso centraliza logging e validação. Ele não é um broker e não atravessa processos.

PERGUNTA 27

MediatR não é excesso para cinco endpoints?

RESPOSTA

Isoladamente poderia ser. Neste teste, CQRS/MediatR é obrigatório e existem dois usos concretos do pipeline. Mesmo assim manteve handlers diretos, sem base classes ou abstrações adicionais.

PERGUNTA 28

Qual é a ordem dos behaviors?

RESPOSTA

Logging foi registrado antes de Validation. Assim, até comandos inválidos têm nome e duração observáveis; o logging captura e registra a falha, mas dados sensíveis de login continuam omitidos. Depois da validação, o handler só recebe uma entrada válida do ponto de vista do contrato.

PERGUNTA 29**Por que validação também no domínio?****RESPOSTA**

FluentValidation valida a entrada e gera detalhes por campo. O Domain protege estado e regras mesmo quando for chamado por job, fila, teste ou outro adapter que não passe pelo pipeline HTTP.

PERGUNTA 30**Por que usar exceções para fluxo esperado?****RESPOSTA**

O escopo é pequeno e um handler global oferece endpoints muito claros. O trade-off é usar exceções em erros esperados. Em um domínio com grande volume de falhas previstas, eu avaliaria um tipo `Result<T>` discriminado para tornar os resultados explícitos sem exceptions.

Persistência

PERGUNTA 31

Por que repositório específico?

RESPOSTA

`IOrderRepository` comunica operações necessárias ao agregado: adicionar, buscar com tracking, buscar read-only, paginar e salvar. Um `IRepository<T>` com CRUD genérico esconderia intenções, vazaria `IQueryable` ou cresceria sem coesão.

PERGUNTA 32

Por que `GetAsync` e `GetReadOnlyAsync` separados?

RESPOSTA

Commands precisam de tracking para que o EF detecte a mudança de status. Queries não alteram a entidade e usam `AsNoTracking`, reduzindo memória e trabalho do change tracker.

PERGUNTA 33

Por que paginação acontece no repositório?

RESPOSTA

`Count`, `OrderBy`, `Skip` e `Take` precisam ser traduzidos e executados pelo banco. Retornar todos os pedidos e paginar em memória seria incorreto para volume real. A Application ainda define a intenção da página e retorna o contrato paginado.

PERGUNTA 34

Por que SQLite?

RESPOSTA

É requisito, tem setup mínimo e permite migrations e integração reais. Para produção concorrente, backup, replicação e observabilidade do banco, eu escolheria PostgreSQL ou outro banco servidor.

PERGUNTA 35**Por que migrations no startup?****RESPOSTA**

É uma exigência e torna avaliação local/Docker simples. Em produção com múltiplas instâncias, eu separaria migrations em job ou etapa de deploy com controle de concorrência e rollback planejado.

PERGUNTA 36**Por que não há uma transação explícita nos handlers?****RESPOSTA**

Cada command atual executa um único `SaveChangesAsync`; o EF Core já o envolve em transação quando necessário. Eu adicionaria transação explícita quando um caso de uso envolvesse múltiplos saves ou recursos, lembrando que banco e broker exigiriam outbox em vez de uma falsa transação distribuída.

PERGUNTA 37**Onde está a unidade de trabalho?****RESPOSTA**

O `DbContext` já é a unidade de trabalho. `SaveChangesAsync` confirma as alterações de um caso de uso. Não criei outra abstração genérica apenas para renomear esse comportamento.

API, autenticação e segurança

PERGUNTA 38

Por que `ProblemDetails`?

RESPOSTA

É um formato HTTP conhecido e consistente. O frontend pode tratar `status`, `title`, `detail` e, em validações, o dicionário `errors`, sem depender de uma mensagem livre diferente por endpoint.

PERGUNTA 39

Por que `201 Created` na criação?

RESPOSTA

Um novo recurso foi criado. Além do corpo, a API envia `Location` apontando para `/api/orders/{id}`. Isso segue a semântica HTTP e permite ao cliente localizar o recurso sem montar a rota por convenção.

PERGUNTA 40

Por que `PATCH` e `204` no cancelamento?

RESPOSTA

O cancelamento altera parcialmente o estado do recurso, portanto `PATCH` corresponde ao contrato pedido. Como não há necessidade de devolver outra representação, sucesso retorna `204`. Se o produto precisasse mostrar o pedido atualizado imediatamente, retornar `200` com o recurso também seria uma decisão defensável, desde que documentada.

PERGUNTA 41

Por que regra de negócio inválida retorna `409` e não `400`?

RESPOSTA

O identificador e a forma da requisição são válidos; o conflito ocorre com o estado atual do recurso. Um pedido já confirmado ou cancelado não aceita a transição, então `409 Conflict` comunica melhor o problema ao cliente.

PERGUNTA 42

Por que existe `Confirmed` no domínio sem endpoint de confirmação?

RESPOSTA

O enum faz parte do domínio solicitado, mas o teste não pediu a operação de confirmar. Mantive a transição no agregado para o modelo não ficar incoerente, porém evitei acrescentar um endpoint fora do escopo. A extensão é simples e está descrita na seção de exercícios ao vivo.

PERGUNTA 43

Por que o health check é anônimo?

RESPOSTA

Orquestradores e balanceadores precisam verificar o processo sem administrar JWT. O endpoint não expõe dados de negócio. Em uma rede pública, acesso e quantidade de detalhes ainda poderiam ser restritos pela infraestrutura.

PERGUNTA 44

Como os erros são mapeados?

RESPOSTA

- `ValidationException` -> 400 com `HttpValidationProblemDetails`;
- credenciais fixas inválidas -> 401;
- `NotFoundException` -> 404;
- `DomainException` -> 409;
- erro desconhecido -> 500 sem detalhe interno e com log de stack trace.

Um JWT ausente/inválido é recusado pelo middleware Bearer antes do endpoint e pode retornar 401 sem corpo. O cliente não deve assumir que todo 401 contém JSON.

PERGUNTA 45

Como o JWT é validado?

RESPOSTA

Issuer, audience, assinatura e expiração são obrigatórios. A chave HMAC precisa ter pelo menos 32 caracteres. O token inclui `sub`, `email` e `jti`, expira em 60 minutos por padrão e aceita clock skew de 30 segundos.

PERGUNTA 46**Por que comparar a senha em tempo constante se ela é fixa?****RESPOSTA**

É uma defesa simples que reduz vazamento por timing e demonstra cuidado. Isso não transforma senha em memória em uma solução produtiva; produção exigiria IdP ou hash apropriado, salt, políticas e proteções contra brute force.

PERGUNTA 47**Como CORS foi tratado?****RESPOSTA**

Existe uma allowlist configurável de origens locais. Headers e métodos são liberados apenas para essas origens. Não usei `AllowAnyOrigin`, e Bearer não requer `AllowCredentials`. Em produção a lista seria substituída pelos domínios oficiais.

PERGUNTA 48**CORS protege a API?****RESPOSTA**

Não contra clientes não-browser. CORS é uma política aplicada pelo navegador. Segurança real vem de autenticação, autorização, HTTPS, validação, rate limiting e demais controles do servidor.

PERGUNTA 49**Por que expor Swagger em Production no Compose?****RESPOSTA**

O Compose é um ambiente de avaliação e a interface facilita a análise do teste. Em produção pública, eu desabilitaria o Swagger ou o colocaria atrás de autenticação e rede restrita.

Testes

PERGUNTA 50

O que os unit tests cobrem?

RESPOSTA

- todos os cinco handlers;
- regras e transições do agregado;
- validators de login, criação, ids e paginação;
- `ValidationBehavior`, inclusive garantia de não chamar o próximo delegate ao falhar;
- `LoggingBehavior`, inclusive redaction e rethrow;
- mapeamento de itens, totais, paginação e relógio.

PERGUNTA 51

O que os testes de integração cobrem?

RESPOSTA

- Swagger;
- preflight CORS;
- rota protegida sem token;
- login inválido;
- validações de criação e paginação;
- `404` ;
- fluxo autenticado de criar, consultar, listar e cancelar;
- segundo cancelamento retornando `409` .

PERGUNTA 52

Por que não usar o provider InMemory do EF?

RESPOSTA

Ele não reproduz semântica relacional, SQL, constraints e peculiaridades do SQLite. Usar um arquivo SQLite isolado testa o mesmo provider da aplicação e mantém independência entre execuções.

PERGUNTA 53**Por que fake manual em vez de Moq/NSubstitute?****RESPOSTA**

As interfaces são pequenas e o fake tem comportamento legível. Ele também permite verificar persistência contando `SaveChangesAsync`. Um framework de mocks seria útil se as colaborações fossem numerosas ou difíceis de implementar, mas aqui adicionaria dependência sem simplificar.

PERGUNTA 54**Cobertura alta garante qualidade?****RESPOSTA**

Não. Cobertura mostra código executado, não qualidade dos asserts. Priorizei regras, transições, efeitos e contratos. Os aproximadamente 95% de linhas e 100% de branches são uma consequência dos cenários relevantes, não o objetivo isolado.

PERGUNTA 55**Os testes podem interferir entre si?****RESPOSTA**

Cada cliente HTTP é criado por teste e cada fixture usa um arquivo de banco exclusivo. Testes da mesma classe não dependem de ordem. O arquivo e auxiliares WAL/SHM são removidos ao descartar a factory.

Logging, tracing e operação

PERGUNTA 56

O que é registrado pelo Serilog?

RESPOSTA

O ciclo HTTP e cada command/query com duração. Requests comuns e respostas são estruturados. `LoginCommand` implementa `ISensitiveRequest`, então e-mail, senha e token não aparecem nesses logs.

PERGUNTA 57

Qual a diferença entre Serilog e OpenTelemetry aqui?

RESPOSTA

Serilog produz logs estruturados. OpenTelemetry cria traces/activities da requisição. Em produção, ambos teriam correlação e seriam enviados para um backend, mas resolvem sinais de observabilidade distintos.

PERGUNTA 58

O health check é suficiente?

RESPOSTA

Para o teste, sim: indica que o processo responde. Em produção eu separaria liveness de readiness e incluiria dependências essenciais, com cuidado para não sobrecarregar o banco.

Docker e entrega

PERGUNTA 59

O que há de relevante no Dockerfile?

RESPOSTA

- build multi-stage da API e build multi-stage separado do frontend;
- restore separado para aproveitar cache;
- runtime menor que a imagem de SDK;
- usuário `app` não-root;
- somente `/app/data` precisa de escrita;
- porta interna `8080` explícita, publicada como `8082` pelo Compose para evitar conflitos no host;
- frontend compilado em Node e servido por Nginx na porta `3000`, com proxy de mesma origem para a API.

PERGUNTA 60

Por que volume para SQLite?

RESPOSTA

Sem volume, os pedidos desapareceriam ao recriar o contêiner. O volume `orders-data` desacopla o arquivo do ciclo de vida do contêiner.

PERGUNTA 61

Por que SonarQube está em profile?

RESPOSTA

SonarQube consome recursos e não é necessário para executar a API. O profile `quality` mantém o caminho normal leve, mas deixa a ferramenta disponível para avaliação.

Integração frontend

PERGUNTA 62

Existe uma interface pronta para avaliar o fluxo?

RESPOSTA

Sim. O diretório `frontend` contém uma aplicação React/Vite que faz login, lista, pesquisa na página, pagina, cria, consulta e cancela pedidos reais. `docker compose up --build` sobe API e frontend; a interface fica em `http://localhost:3000` e o Nginx encaminha `/auth` e `/api` para o serviço interno. O layout é responsivo e usa a linguagem visual da NTT DATA inferida do material e validada contra o site oficial.

PERGUNTA 63

O que o frontend precisa saber primeiro?

RESPOSTA

Base URL, login, armazenamento/expiração do token, header Bearer, contratos, paginação e `ProblemDetails`. O README contém tipos TypeScript e um cliente `fetch` completo.

PERGUNTA 64

Como o frontend deve tratar dinheiro?

RESPOSTA

Deve exibir o total retornado pela API com `Intl.NumberFormat` e não recalculá-lo como fonte de verdade. O JSON usa número; o frontend deve evitar transformações que percam centavos. O domínio atual não possui moeda, uma limitação explicitamente documentada.

PERGUNTA 65

Como tratar o segundo cancelamento?

RESPOSTA

Desabilitar o botão quando o estado conhecido não for `Pending`, mas ainda tratar `409` porque pode haver mudança concorrente. Ao receber `409`, recarregar o pedido e apresentar `detail`.

PERGUNTA 66

Onde guardar o token?

RESPOSTA

O exemplo usa `sessionStorage` apenas por simplicidade. Para um produto público eu faria modelagem de ameaça. Uma SPA precisa de prevenção forte contra XSS; um BFF com cookie `HttpOnly/Secure` e proteção CSRF pode reduzir exposição do token ao JavaScript.

Extensões prováveis ao vivo

PERGUNTA 67

Como adicionar `PATCH /api/orders/{id}/confirm`?

RESPOSTA

1. Criar `ConfirmOrderCommand` e validator de GUID.
2. Criar handler que busca com tracking, lança `NotFoundException`, chama `order.Confirm()` e salva.
3. Mapear o endpoint protegido e seus status `204`, `404`, `409`, `401`.
4. Adicionar unit tests para sucesso, inexistente e status inválido.
5. Adicionar um cenário de integração.

O método de domínio já existe, mas eu ainda passaria pelo command para manter o fluxo consistente.

PERGUNTA 68

Como adicionar filtro por status na listagem?

RESPOSTA

1. Adicionar `OrderStatus?` à query e validar valores aceitos.
2. Alterar a assinatura específica de `GetPageAsync`.
3. Aplicar `Where` antes de `Count`, `Skip` e `Take`.
4. Expor query string opcional no endpoint.
5. Testar filtro, paginação e combinação sem resultados.

Eu não retornaria `IQueryable` para a Application porque isso vazaria detalhes do provider.

PERGUNTA 69

Como adicionar idempotência na criação?

RESPOSTA

Aceitaria `Idempotency-Key`, persistiria chave, status e resposta dentro da mesma transação do pedido, criaria índice único e devolveria a resposta anterior para repetição equivalente. Definiria retenção e comportamento quando a mesma chave viesse com payload diferente.

PERGUNTA 70**Como publicar um evento `OrderCreated` com segurança?****RESPOSTA**

Criaria evento de domínio e gravaria uma mensagem outbox na mesma transação do pedido. Um worker publicaria e marcaria a mensagem como processada. Consumidores seriam idempotentes. Publicar direto após `SaveChanges` cria uma janela de perda e não seria suficiente.

PERGUNTA 71**Como trocar SQLite por PostgreSQL?****RESPOSTA**

Mudaria o provider/connection string na Infrastructure, geraria migrations adequadas e executaria testes contra PostgreSQL real, idealmente com container efêmero. Application e Domain não deveriam mudar.

PERGUNTA 72**Como versionar a API?****RESPOSTA**

Primeiro confirmaria a necessidade de breaking changes. Adicionaria versionamento na camada API, manteria DTOs por versão e reutilizaria commands/queries quando a semântica fosse igual. Não colocaria versão no Domain.

Perguntas difíceis e respostas honestas

PERGUNTA 73

O que está deliberadamente fora do escopo?

RESPOSTA

Cadastro real de usuários, refresh token, moeda, estoque, pagamento, confirmação via endpoint, edição de pedidos, exclusão, auditoria, concorrência otimista, idempotência, API versioning, cache, mensageria e observabilidade externa. Foram evitados para manter foco no teste.

PERGUNTA 74

Qual é o principal risco técnico atual?

RESPOSTA

Para uso concorrente, a ausência de concurrency token e o SQLite. Para segurança, as credenciais e chaves fixas de avaliação. Para evolução de domínio, a ausência de moeda e de uma política explícita de arredondamento.

PERGUNTA 75

O que você não faria mesmo com mais tempo?

RESPOSTA

Não criaria microserviços, repository genérico, event bus, AutoMapper, base handler ou abstrações de relógio/banco apenas para aumentar a quantidade de padrões. Eu adicionaria essas peças somente quando resolvessem um problema demonstrável.

PERGUNTA 76

Se CQRS não fosse obrigatório, você ainda usaria?

RESPOSTA

Eu manteria a separação conceitual dos casos de uso, mas avaliaria se MediatR agrega valor. Como há logging e validação transversais, ele continua defensável. Em uma API ainda menor, chamadas diretas a application services específicos também poderiam ser mais simples.

PERGUNTA 77**Por que não usar AutoMapper?****RESPOSTA**

Há poucos modelos e o mapeamento explícito é curto, navegável e verificado pelo compilador. Uma dependência de mapeamento seria justificável com muitos modelos repetitivos, desde que sua configuração também fosse testada.

PERGUNTA 78**Por que não persistir o total para acelerar listagens?****RESPOSTA**

O teste prioriza consistência e exige cálculo no domínio. Sem evidência de gargalo, duplicar o dado é prematuro. Se necessário, eu criaria uma projeção de leitura ou persistência controlada, mantendo a regra e a atualização atômica.

Roteiro para demonstrar ao avaliador

1. Mostrar a direção das referências entre projetos.
2. Abrir `Order.Create`, `Cancel` e `TotalAmount` para provar onde estão as regras.
3. Abrir um endpoint e seu handler para mostrar que HTTP não contém regra de negócio.
4. Mostrar `ValidationBehavior` e `LoggingBehavior`.
5. Mostrar `IOrderRepository` e a implementação com tracking/no-tracking.
6. Rodar `dotnet test OrderManagement.slnx`.
7. Abrir o Swagger, fazer login, criar, listar, consultar e cancelar.
8. Repetir o cancelamento para demonstrar o `409` esperado.
9. Explicar os trade-offs e as mudanças necessárias em produção.

Comandos úteis durante a entrevista

BASH

```
dotnet build OrderManagement.slnx --configuration Release
dotnet test OrderManagement.slnx --configuration Release
dotnet format OrderManagement.slnx --verify-no-changes
dotnet ef migrations has-pending-model-changes \
  --project src/OrderManagement.Infrastructure \
  --startup-project src/OrderManagement.Api
docker compose config
docker compose up --build
```